# COS10022

Assignment 2 template

Read and respond to each question in this template. Some questions will require you to include a screenshot of your work in KNIME – please ensure that these screenshots are clear to ensure that you get marked correctly for your work.

## Step 1: Data cleaning – 70 marks

Follow the instructions to clean the data and answer questions. If any of the nodes you used in the workflow has a random seed, set **3122** to the seed to fix the random state.

1. Our goal is to predict the credit score from the given data. There is/are one (or multiple) attribute(s) which is/are significantly irrelevant to the goal. Exclude the attribute(s) and give a persuasive rationale for that. **(5 marks)**

**The excluded attribute/s is/are:**
**Name**
**The attribute is/are excluded because:**

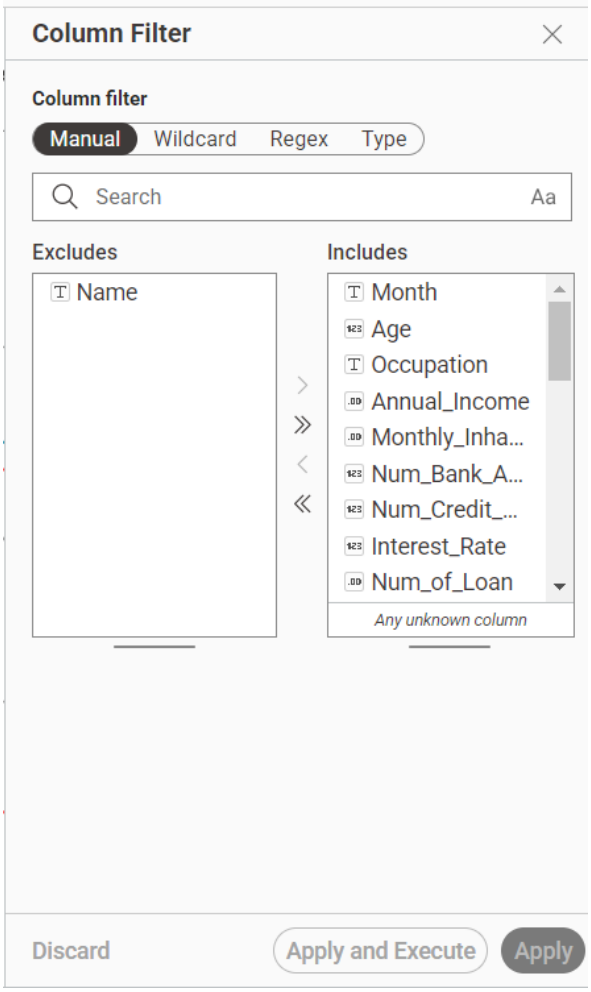
- Irrelevance to credit scoring:
  A person's name does not have to provide any financial, behavioural, or demographic information that influences credit score. Credit scoring models rely on measurable financial attributes such as income, loan history, credit utilisation, and payment behaviour – not personal identifiers.

- Excessive number of unique values:
  Using the Value Counter node, the Name attribute contains 10,137 unique values, far exceeding the Naïve Bayes learner's limit of 600 unique nominal values. Including this attribute would cause the model to fail or produce unstable results.

- Privacy and model efficiency:
  Names are personally identifiable information and do not contribute to predictive performance. Removing them improves model efficiency and protects privacy.



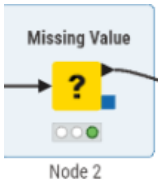

**To exclude the Name attribute. I used Column Filter.**

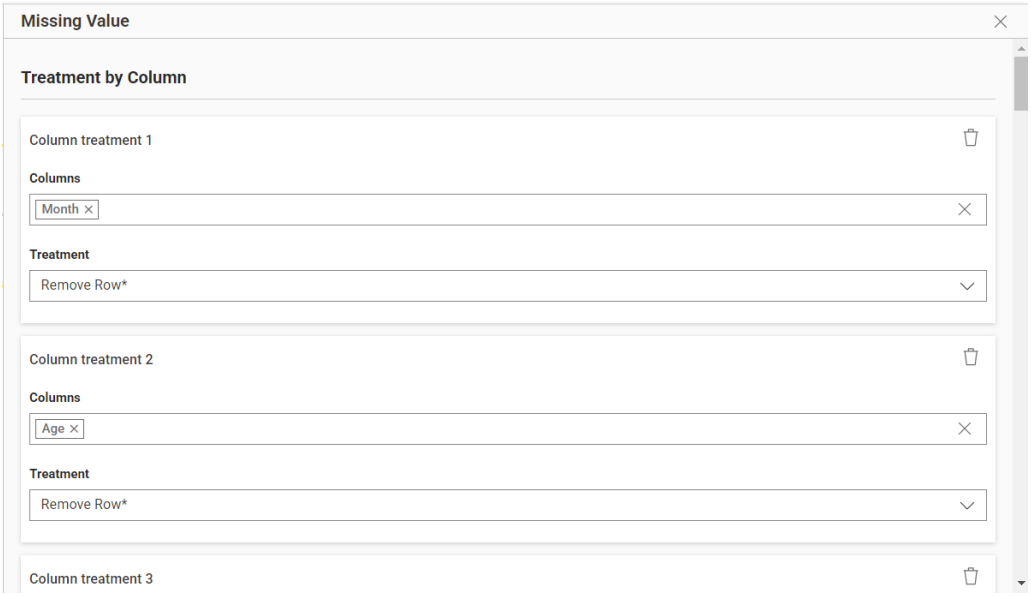



2. After removing the selected attribute/s, let's start to remove tuples containing missing values. Remove tuples only if any of the attributes listed below have missing values: 'Month', 'Age', 'Occupation', 'Annual_Income', 'Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate', 'Num_of_Loan', 'Delay_from_due_date', 'Changed_Credit_Limit', 'Credit_Mix', 'Outstanding_debt', 'Credit_Utilisation_Ratio', 'Credit_History_Age', 'Payment_of_Min_Amount', 'Total_EMI_per_month', 'Amount_invested_monthly', and 'Payment_Behaviour'. Moreover, some tuples with infeasible values in the attributes, such as 'Monthly_Inhand_Salary' < 0, 'Num_Bank_Accounts' < 0, 'Num_Credit_Card' < 0, and 'Changed_Credit_Limit' contains '_', should also be removed.

   List the node(s) (in sequence) **(2.5 marks)** and the corresponding command(s) used in this process **(2.5 marks)**. **(2.5 + 2.5 = 5 marks)**

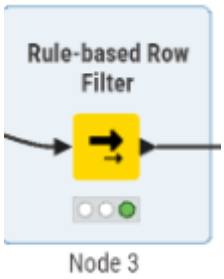
**The used node is:**
**Missing Value (Node 2)**




**Which shows configuration.**

**Rule-based Row Filter (Node 3)**



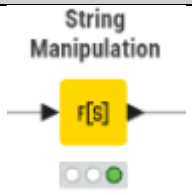
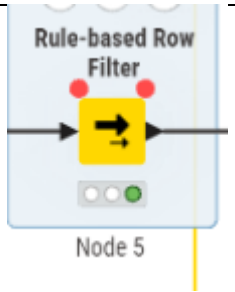

Moreover, tuples that contain infeasible values will be also excluded, such as:

1. 'Monthly_Inhand_Salary' < 0
2. 'Num_Bank_Accounts' < 0
3. 'Num_Credit_Card' < 0
4. 'Changed_Credit_Limit' contains '_'

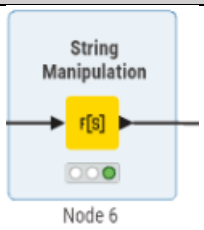
**The used commands are listed are:**

**$Monthly_Inhand_Salary$ < 0 => TRUE**
**$Num_Bank_Accounts$ < 0 => TRUE**
**$Num_Credit_Card$ < 0 => TRUE**
**$Changed_Credit_Limit$ MATCHES "_" => TRUE**

3. Check for the 'Age' attribute to eliminate symbols that are not numbers to recover the data into the usual number format. Moreover, drop the tuples whose 'Age' value is lower than or equal to 0 or greater than 120. List the node(s) (in sequence) **(2.5 marks)** and the corresponding command(s) used in this process **(2.5 marks)**. **(2.5 +2.5 = 5 marks)**

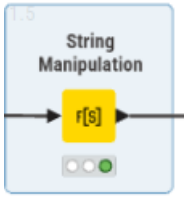
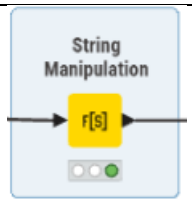
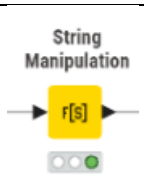
| Sequence | Node | Command |
|---|---|---|
| 1 |  | toInt(regexReplace($Age$,"[^0-9.-]","")) |
| 2 |  | $Age$ > 0 AND $Age$ <= 120 => TRUE |

4. Remove the non-numerical symbol in the 'Annual_Income' column and convert it to the double format. List the node(s) (in sequence) **(2.5 marks)** and the corresponding command(s) used in this process. **(2.5 marks)**. **(2.5 +2.5 = 5 marks)**

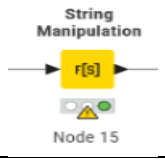
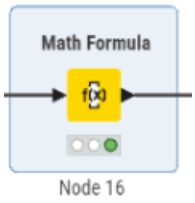
| Sequence | Node | Command |
|---|---|---|
| 1 |  | toDouble(regexReplace($Annual_Income$,"[^0-9.-]","")) |

5. Complete the following steps:
    - Convert the '_____' in the 'Occupation' attribute to Null. Please note that Null is different from an empty string.
    - Remove the non-numerical symbol in 'Num_of_Loan' and convert it to integer data type.
    - Take absolute values of attributes 'Num_Bank_Accounts' and 'Num_Credit_Card'.
    - Set values to 0 for the 'Num_of_Loan' attribute if the original values are negative.
    - Remove the non-numerical symbol in 'Num_of_Delayed_payment' and convert it into integer format.
    - Set the 'Credit_Mix' value to 'Unknown' if the original value is '_'.
    - Remove the non-numerical symbol in 'Outstanding_Debt' and convert it into the double format.

    Then list the node(s) (in sequence) and the corresponding command(s) used in this process. **(10 marks)**

| Sequence | Node | Command |
|---|---|---|
| 1 |  | replace($Occupation$,"_______" , null) |
| 2 |  | toInt(regexReplace($Num_of_Loan$,"[^0-9.-]","")) |
| 3 |  | abs($$CURRENT_COLUMN$$) |
| 4 |  | if($Num_of_Loan$ < 0, 0, $Num_of_Loan$) |
| 5 |  | toInt(regexReplace( $Num_of_Delayed_Payment$,"[^0-9.-]","")) |
| 6 |  | replace($Credit_Mix$, "_", "Unknown" ) |
| 7 |  | toDouble(regexReplace($Outstanding_Debt$,"[^0-9.-]","")) |

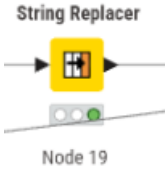
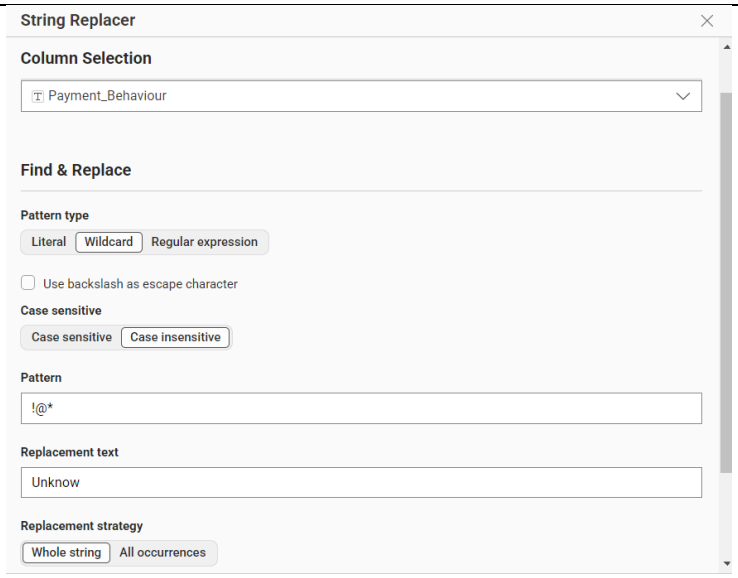
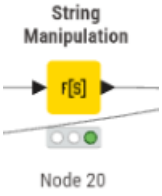
6. Convert the 'Credit_History_Age' to the count of months and store it in the integer format. For example, if the original value from a tuple is '22 Years and 1 Months', the value will be 265 after the conversion (22 * 12 + 1 = 265). Store the converted result in a new attribute called 'Total_CHA'. List the node(s) (in sequence) and the corresponding command(s) used in this process. **(10 marks)**

| Sequence | Node | Command |
|---|---|---|
| 1 |  | toInt(strip(substr($Credit_History_Age$, 0, 2))) |
| 2 |  | toInt(strip(substr($Credit_History_Age$, indexOf($Credit_History_Age$, "d") + 1, 3)<br>)) |
| 3 |  | $Years$ * 12 + $Months$ |

7. Complete the following steps:
   - Remove the non-numerical symbol in 'Amount_invested_monthly' and convert it to the double format.
   - Set the value to 'Unknown' if the original value in 'Payment_Behaviour' attribute starts with '!@'.
   - Remove the non-numerical symbol in 'Monthly_Balance' and convert it to the double format.
   - Convert 'Changed_Credit_Limit' into the double format.

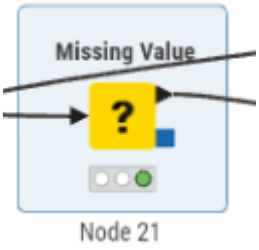
   List the node(s) (in sequence) and the corresponding command(s) used in this process. **(5 marks)**

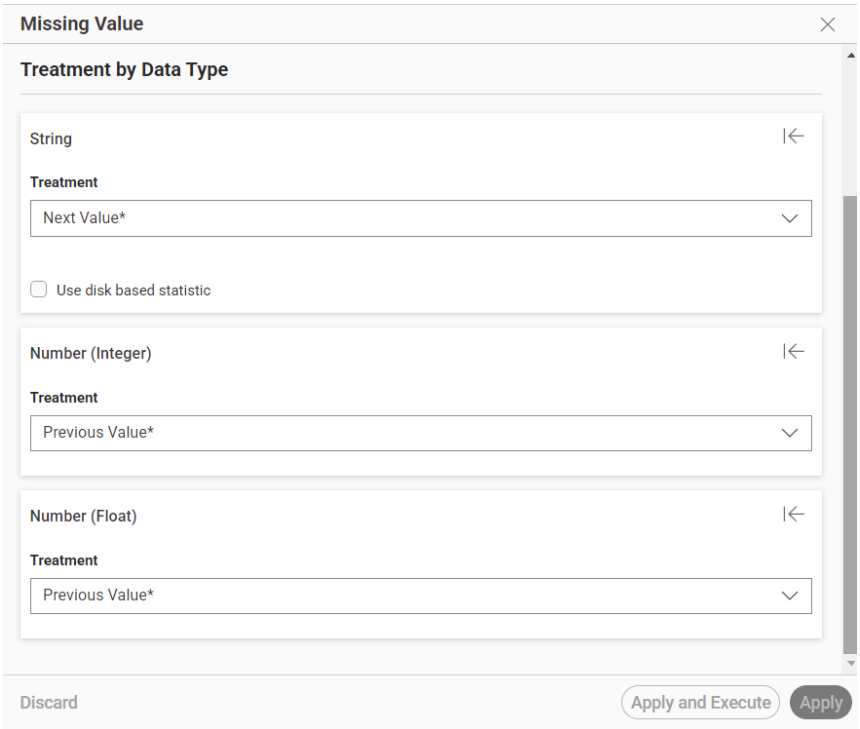
| Sequence | Node | Command |
|---|---|---|
| 1 |  | toDouble(regexReplace($Monthly_Balance$,"[^0-9.-]","")) |

| | | |
|---|---|---|
| 2 |  | toDouble(regexReplace($Amount_invested_monthly$,"[^0-9.-]",""))<br>This node sets the value to “Unknow” in the “Payment_Behaviour” column if the original value can begin with “!@”. The “*” wildcard is used to match the pattern in the specified order. |
| 3 |  |  |
| 4 |  | toDouble($Changed_Credit_Limit$) |

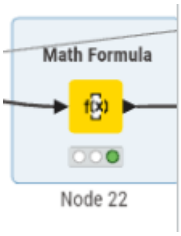
8. Complete the following steps:
    - Use the 'Missing Value' node and use the 'Next Value*' to replace missing values in all string type attributes.
    - Use the 'Previous Value*' in the same node to replace missing values in any numerical format.
    - If the value of 'Monthly_Balance' is negative, replace the value with 0. Screenshot the pop-up window with the correct settings.

    Then, provide a screenshot of the pop-up window with the correct settings. **(5 marks)**




**The node can replace missing values in all string attributes with "Next Value*" and in all numeric attributes with "Previous Value*".**

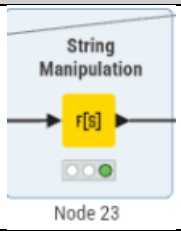
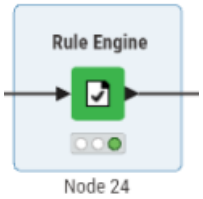



**Expression:**

**if($Monthly_Balance$ < 0, 0, $Monthly_Balance$)**

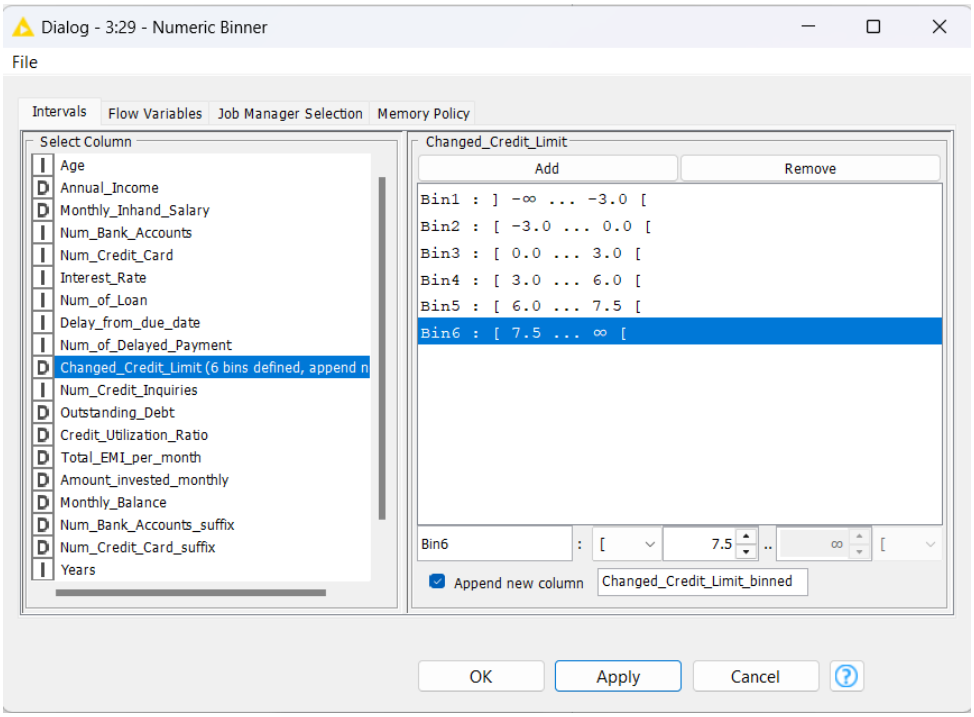
**This expression modifies the "Monthly_Balance column based on a condition: if "Monthly_Balance" is less than 0, it is set to 0. Otherwise, if it is 0 or higher, the original value is retained.**

9. Simplify the 'Type_of_Loan' attribute. If the original content has more than one type separated by a comma, keep only the first part. Otherwise, keep the full description if there is no comma included. For example, 'Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan' will become 'Auto Loan', 'Credit-Builder Loan' will still be 'Credit-Builder Loan', and 'Not Specified, Auto Loan, and Student Loan' will become 'Not Specified' after the process. List the node(s) (in sequence) **(5 marks)** and the corresponding command(s) used in this process **(5 marks)**. **(5 + 5 = 10 marks)**

| Sequence | Node | Command |
|---|---|---|
| 1 |  | substr( $Type_of_Loan$,0,indexOfChars($Type_of_Loan$,",")) |
| 2 |  | substr ( $tmp_ToL$ LIKE "" => $Type_of_Loan$ TRUE => $tmp_ToL$ |

10. Bin the 'Changed_Credit_Limit' attribute with six bins of ranges: [−∞, −3.0), [−3.0, 0), [0, 3.0), [3.0, 6.0), [6.0, 7.5), and [7.5, ∞) and put the result into a new attribute called 'Changed_Credit_Limit_binned'.

    Screenshot the pop-up window with the correct settings of your binner. **(5 marks)**




11. Remove all temporarily created or useless attributes. Use the 'Feature Selection Loop Start (1:1)' node to select the feature. The class label should be excluded from the features in the feature selection node. The Genetic Algorithm is specified to be the feature selection strategy with default population size and the maximum number of generations. Again, **3122** should be used as the static random seed. After selecting features, shuffle the data with seed **3122**. The data should be partitioned by 'Linear sampling', with 75% data in the training set and 25% in the test set.

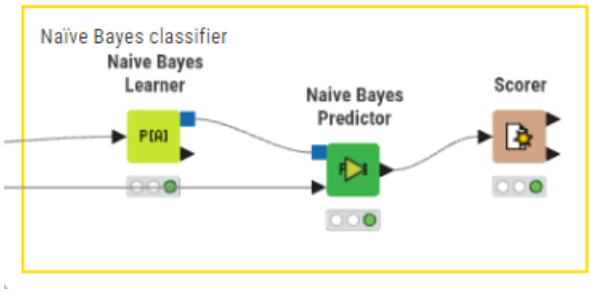
    How many tuples **(2.5 marks)** and attributes (excluding the class label) **(2.5 marks)** are in the training set at the end? **(2.5 + 2.5 = 5 marks)**

    **There are 68,196 tuples and 12 attributes.**

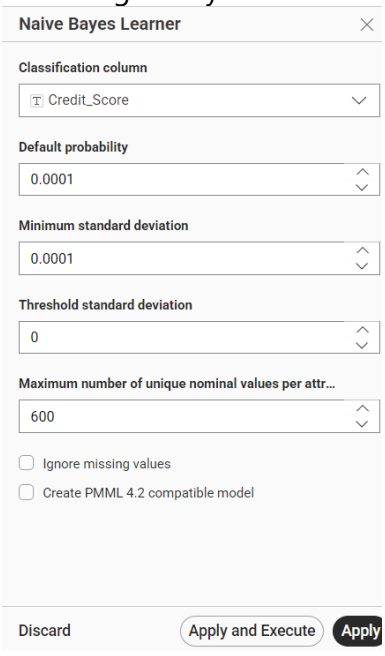
## Step 2: Naïve Bayes Model – 15 marks

Build a Naïve Bayes classifier using the training and test sets created in the previous task. Answer the following questions after completing the model training and test.

1. Provide a screenshot of the Naïve Bayes classifier in the KNIME workflow. You can take the screenshot starting from the portioning node output to the end of the Naïve Bayes classifier part scorer. **(2.5 marks)**




2. The default probability should be 0.0001, the minimum standard deviation is 0.0001, the threshold standard deviation is 0, and the maximum number of unique nominal values per attribute should be set to 600 in the classifier. Screenshot the setting dialogue of your Naïve Bayes Learner. **(2.5 marks)**




The Naïve Bayes Learner was configured using the required parameter values.
Both the default probability and the minimum standard deviation were set to 0.0001. The threshold standard deviation was set to 0, and the limit for the maximum number of unique nominal values per attribute was adjusted to 600. These settings ensure the classifier operates within the constraints specified for the model.

3. Screenshot the confusion matrix and the Accuracy statistics of the test result. **(2.5 marks)** If the bank wants to minimise the risk of lending money to customers, the 'Good' in 'Credit_Score' should be the major target. Based on the current result, does the classifier perform satisfactorily? Include your answer to this below your screenshot. **(2.5 marks) (2.5 +2.5 = 5 marks)**

Confusion matrix (Table)

Rows: 3 | Columns: 3

| # | RowID | Good Number (Integer) | Standard Number (Integer) | Poor Number (Integer) |
|---|---|---|---|---|
| 1 | Good | 3301 | 709 | 93 |
| 2 | Standa | 2639 | 7288 | 2156 |
| 3 | Poor | 958 | 1999 | 3589 |

Accuracy statistics (Table)

Rows: 4 | Columns: 11

| # | RowID | TruePositives Number (Inte... | FalsePositives Number (Inte... | TrueNegatives Number (Inte... | FalseNegativ... Number (Inte... | Recall Number (Float) | Precision Number (Float) | Sensitivity Number (Float) | Specificity Number (Float) | F-measure Number (Float) | Accuracy Number (Float) | Cohen's kappa Number (Float) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Good | 3301 | 3597 | 15032 | 802 | 0.805 | 0.479 | 0.805 | 0.807 | 0.6 | | |
| 2 | Standa | 7288 | 2708 | 7941 | 4795 | 0.603 | 0.729 | 0.603 | 0.746 | 0.66 | | |
| 3 | Poor | 3589 | 2249 | 13937 | 2957 | 0.548 | 0.615 | 0.548 | 0.861 | 0.58 | | |
| 4 | Overall | | | | | | | | | | 0.624 | 0.41 |

**Does the classifier perform satisfactorily?**

**No – the classifier does not perform satisfactorily for minimising lending risk.**

**Analysis:**

- Overall Accuracy (62.4%)
  The model correctly classifies about 62% of cases. While this is better than random guessing, it is not sufficient for a high-risk domain like lending, where misclassifying risky customers as "Good" can cause financial loss.
- Cohen's Kappa (0.41)
  This indicates moderate agreement between predictions and true labels. For credit scoring, banks typically require substantial agreement (0.6+).
  A kappa of 0.41 shows the model is not reliable enough.
- "Good" Class:
- Recall = 0.805
  The model correctly identifies 80.5% of truly "Good" customers.
  This is decent – the bank is able to find most trustworthy customers.
- Precision = 0.479
  This is the major problem.
  Only 47.9% of customers predicted as "Good" are actually Good.
  More than half of the predicted "Good" customers are actually Standard or Poor.
  This means the bank would mistakenly approve loans to many risky customers, increasing financial exposure.
- False Positives (3597)
  These are customers predicted as "Good" but actually Standard or Poor.
  This number is too high for safe lending.
- False Negatives (802)
  These are truly "Good" customers predicted as Standard or Poor.

This means that bank misses trustworthy customers, losing potential business.

Conclusion
The model does not achieve the level of reliability needed for decreasing lending risk. Even though its overall accuracy is 62.4% and it identifies many "Good" customers (recall of 0.805), the precision for the "Good" class is very low at 0.479. This means a large portion of customers predicted as "Good" are actually Standard or Poor, as shown by the high number of false positives (3597). Misclassifying these higher-risk customers as low-risk creates a significant financial risk for the bank.
Because of this, the classifier is not suitable for safe credit-approval decisions, and improvements in precision and class discrimination are necessary.

4. Which measurement should we look at to interpret your conclusion in this case? **(5 marks)**

The key measurement to interpret the classifier's suitability for minimising lending risk is Precision for the "Good" class.

Precision can directly measure how many customers predicted as "Good" are actually low-risk. This is critical for lending decisions because false positives – cases where "Standard" or "Poor" customers are incorrectly classified as "Good" – expose the bank to financial risk. A high precision value can mean the model rarely mislabels risky customers as "Good", which is essential for safe lending.

In the confusion matrix, Precision is calculated as:

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$$

Where:

- True Positives (TP) is the number of "Good" customers correctly identified.
- False Positives (FP) is the number of "Standard" or "Poor" customers incorrectly identified as "Good"

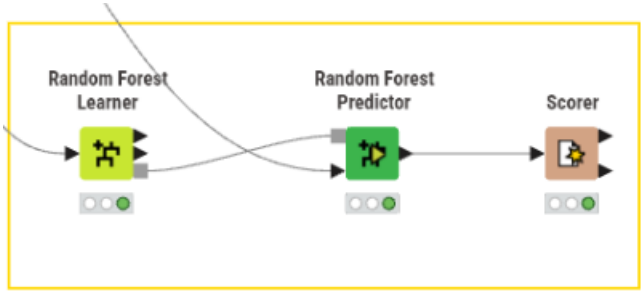
In the model, the Precision for "Good" is only 0.479, meaning more than half of the customers predicted as "Good" are actually not Good. This low precision indicates that the classifier frequently misclassifies higher-risk customers as low-risk, making it unsuitable for the bank's goal of minimising lending risk.

Other metrics such as Recall, Specificity, and F-measure provide useful context, but Precision is the most important metric because it directly reflects the model's ability to avoid risky lending decisions.

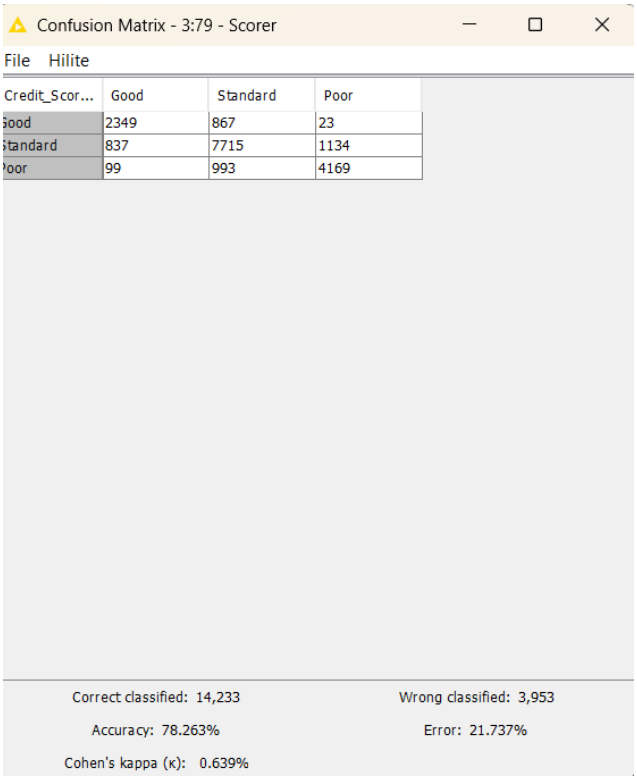
## Step 3: Random Forest Model – 15 marks

Build a random forest classifier using the training and test sets created in the previous task. Answer the following questions after completing the model training and test. Use the information gain ratio as the split criterion and 3122 as the static random seed to build the random forest model.

1. Give a screenshot of the random forest classifier in the KNIME workflow. You can take the screenshot starting from the portioning node output to the end of the Naïve Bayes classifier part scorer. **(2.5 marks)**




2. Screenshot the confusion matrix and the Accuracy statistics of the test result. **(2.5 marks)**



Accuracy statistics (Table)

Rows: 4 | Columns: 11

| # | RowID | TruePosit... Number (... | FalsePosi... Number (... | TrueNega... Number (... | FalseNeg... Number (... | Recall Number (... | Precision Number (... | Sensitivity Number (... | Specificity Number (... | F-measure Number (... | Accuracy Number (... | Cohen's k Number |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Good | 2349 | 936 | 14011 | 890 | 0.725 | 0.715 | 0.725 | 0.937 | 0.72 | | |
| 2 | Standa | 7715 | 1860 | 6640 | 1971 | 0.797 | 0.806 | 0.797 | 0.781 | 0.801 | | |
| 3 | Poor | 4169 | 1157 | 11768 | 1092 | 0.792 | 0.783 | 0.792 | 0.91 | 0.788 | | |
| 4 | Overall | | | | | | | | | | 0.783 | 0.639 |

3. If the bank wants to minimise the risk of lending money to customers, the 'Good' in 'Credit_Score' should be the major target. Compare the measurements between random forest results and Naïve Bayes results. Which model presents a more suitable result? **(2.5 marks)** Which measure should be used to make the comparison? **(2.5 marks) (2.5 +2.5 = 5 marks)**

   **If the bank wants to minimise lending risk, the most important goal is to avoid misclassifying risky customers ("Standard" or "Poor") as "Good".**
   **Therefore, the key measurements to compare between the 2 models is the False Positive Rate (FPR) for the "Good" class.**

| | Naïve Bayes model | Random Forest mode |
|---|---|---|
| **Precision (Good)** | **0.479** | **0.715** |
| **Accuracy** | **0.624** | **0.783** |

1. **False Positive Rate (FPR) — Naïve Bayes**
   **From my Naïve Bayes confusion matrix:**

- False Positives (FP) for "Good" = 3597
- True Negatives (TP) for "Good" = 15032

$$FPR_{NB} = \frac{FP}{FP + TN} = \frac{3597}{3597 + 15032} = 0.1939 \approx 19.4\%$$

2. False Positives Rate (FPR) – Random Forest

From my Random Forest confusion matrix

- False Positives (FP) for "Good" = 936
- True Negatives (TN) for "Good" = 15032

$$FPR_{RF} = \frac{936}{936 + 14011} = 0.0593 \approx 5.93\%$$

3. Comparison – Which model is more suitable?

| Classifier | False Positive Rate (FPR) for "Good" |
|---|---|
| **Naïve Bayes** | 19.4% |
| **Random Forest** | 5.93% |

Random Forest has a much lower False Positive Rate, meaning it is far less likely to incorrectly classify risky customers as "Good."
This can make Random Forest significantly safer for lending decisions.

**4.** Which measure should be used?

The most appropriate measure is:

False Positive Rate (FPR) for the "Good" class

Because:

- FPR directly measures how often the model incorrectly labels risky customers as "Good."
- This is the most dangerous error type of a bank.
- Lower FPR = lower lending risk.

Precision can also be useful, but FPR is the most direct measure of lending risk.

Conclusion:
The Random Forest model is more suitable for minimising lending risk because it has a much lower False Positive Rate (5.93%) compared to Naïve Bayes (19.4%). The best measure for comparing the models in this context it is the False Positive Rate (FPR) for the "Good" class, as it directly reflects how often risky customers are incorrectly classified as low-risk.

4. Which class does the built random forest model perform the best? **(2.5 marks)** What measurement(s) should we look at to find the answer? **(2.5 marks) (2.5 + 2.5 = 5 marks)**

**To identify which class the Random Forest model handles most effectively, we need to compare how well it predicts each category. The most useful metrics for this comparison are Precision, Recall, and F-measure, because they show how accurately and consistently the model recognises each class.**

- Precision tells us how many of the predicted cases for a class are actually correct.
- Recall shows how many true instances of a class the model successfully identifies.
- F-measure combines both Precision and Recall, giving a balanced view of overall performance.

Looking at the Random Forest results:
- Good: Precision = 0.715, Recall = 0.725, F-measure = 0.720
- **Standard: Precision: 0.806, Recall = 0.797, F-measure = 0.801
- Poor: Precision = 0.783, Recall = 0.792, F-measure = 0.788

Among the 3, the Standard class has the strongest overall performance, with the highest Precision, Recall, and F-measure. This indicates that the Random Forest model predicts the Standard category more accurately than the others.

References:


1. Kaggle. (2023). *Kaggle platform*. https://www.kaggle.com/
2. KDD. (2023). *KDD Cup archive*. https://www.kdd.org/kdd-cup
3. KNIME. (2023). *KNIME analytics platform*. https://www.knime.com/
4. Lee, J.-Y. (2017). *Winning data science competitions* [Image]. SAS Blogs. https://blogs.sas.com/content/subconsciousmusings/2017/05/18/stacked-ensemble-models-win-data-science-competitions/
5. AS Watson Group. (2023). *Company homepage*. https://www.aswatson.com/
6. DataCamp. (2023). *Market basket analysis in R*. https://www.datacamp.com/courses/market-basket-analysis-in-r
7. Han, J., Kamber, M., & Pei, J. (2011). *Data mining: Concepts and techniques*. Elsevier.
8. KNIME. (2023). *KNIME analytics platform*. https://www.knime.com/
9. AS Watson Group. (n.d.). *AS Watson Group logo* [Image]. https://www.aswatson.com/
10. Tableau. (2023). *Tableau Public*. https://public.tableau.com/app/discover
11. Todd, S. (2012, March 5). *Phase 1: Innovation analytics*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/03/phase-1-innovation-analytics.html
12. Todd, S. (2012, March 29). *Phase 2: Data preparation*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/03/phase-2-innovation-analytics-data-preparation-1.html
13. Todd, S. (2012, May 3). *Phase 3: Model planning*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/05/phase-3-innovation-analytics-.html
14. Todd, S. (2012, June 4). *Phase 4: Finding boundary spanners*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/06/finding-boundary-spanners.html
15. Todd, S. (2012, June 19). *Phase 5: Global knowledge flight patterns*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/06/phase-5-innovation-analytics-global-knowledge-flight-patterns.html
16. Todd, S. (2012, July 5). *Phase 6: Operationalize*. Information Playground. https://stevetodd.typepad.com/my_weblog/2012/07/phase-6-innovation-analytics-operationalize.html
17. KNIME. (2023a). *KNIME analytics platform*. https://www.knime.com/
18. KNIME. (2023b). *Apriori (3.7) node documentation*. https://hub.knime.com/knime/extensions/org.knime.features.ext.weka_3.7/latest/org.knime.ext.weka37.associator.WekaAssociatorNodeFactory:66f98dbc
19. NodePit. (n.d.). *Apriori (3.7)*. https://nodepit.com/node/org.knime.ext.weka37.associator.WekaAssociatorNodeFactory%23Apriori%20(3.7)
20. Dietrich, D., Heller, B., & Yang, B. (2015). *Data science and big data analytics: Discovering, analyzing, visualizing and presenting data*. Wiley.
21. Gorunescu, F. (2011). *Data mining: Concepts, models and techniques*. Springer.

22. KNIME. (2023). *KNIME analytics platform*. https://www.knime.com/

GenAI use:

The genAI tool used was Copilot. Microsoft (2026) Copilot (2026) [GPT 5.6]. Copilot (microsoft.com)

The prompts(s) I used were

- Clarifying KNIME node settings and workflow steps
- Interpreting confusion matrices and accuracy statistics
- Rephrasing explanations to decrease similarity
- Checking calculations such as precision, recall, and false positive rate
- Improving the clarity and structure of written answers

The output from these prompts was used to

- Understand the results produced by the Naïve Bayes and Random Forest models
- Rewrite sections of the assignment in my own words
- Verify the correctness of formulas and evaluation metrics
- Organise explanations more clearly and logically